Assignment#01 - From Text to Knowledge Graph

Student Guide

At a glance

Type	Individual work: a take-home part plus a short viva
Marks	100 in total: 60 take-home and 40 viva
Covers	Week 4 (knowledge graphs) and Week 5 (ontologies)
Hand in	One zip file or a link to your repository
Due date	3rd November 2026

This guide explains the assignment in plain steps. If anything here seems to disagree with the official assignment handout, ask in class before you start.

1 The Big Idea

You will turn sentences into a small knowledge graph, check the graph, ask it questions, and measure how good it is. You do it all yourself, on data that belongs only to you.



Triple

A fact written as three parts: *subject, predicate, object.* For example: `Nadia Farooq -- bornIn -- Lahore`.

One sentence, start to finish

Step	What happens
Text	"Nadia Farooq was born in Lahore in 1950. She studied at Midlands University."
Triples	(Nadia Farooq, bornIn, Lahore) (Nadia Farooq, educatedAt, Midlands University). Notice that "She" had to be turned back into "Nadia Farooq".
Rules	"bornIn" links a Person to a Place; a person has only one birthplace.
Query	"Who was born in Lahore?"
Score	Compare your triples with Wikidata. Which ones match, which are wrong, which are missing?

2 What You Receive

A folder with your roll number containing two files.

- `ASSIGNMENT_SHEET.md`: your five Wikipedia article titles, the name to give your Fuseki dataset, your personal namespace, and the numbered positions you will analyse in the last part.
- `private_document.txt`: a short biography of an *invented* person, written only for you. It is tricky on purpose: it uses pronouns, nicknames, and a sentence about something that did *not* happen.

Your text collection is your five Wikipedia articles plus your private document. Your results will differ from your classmates', so keep your folder to yourself.

3 The Five Facts You Extract

Predicate	Meaning	Example sentence
`bornIn`	place of birth	"X was born in Lahore."
`educatedAt`	school or university	"X studied at Y."
`receivedAward`	prize or medal	"X received the Y Prize."
`worksAt`	employer	"X joined Y."
`doctoralAdvisor`	PhD supervisor	"X's doctoral advisor was Y."

4 Step by Step

Work in this order. Each step says when you are done.

Step 0 – Set up (5 marks)

Install Python 3.10 or newer, spaCy, rdflib, pySHACL and requests (see Appendix A), and Apache Jena Fuseki (it needs Java 17 or newer). Start Fuseki, open `http://localhost:3030`, and create a **persistent (TDB2)** dataset with the name from your sheet. Create a Git repository and commit as you work: your commit history is part of the marks.

Done when: you have a screenshot of the Fuseki page showing your dataset, and a repository with at least a few commits.

Step 1 – Get the text

Download the plain text of your five Wikipedia articles (Appendix B has a short example) into `data/raw/`, and copy in your `private_document.txt`.

Done when: `data/raw/` holds six text files.

Step 2 – Write the extractor (20 marks)

Write a Python program, `src/extract.py`, that reads the text and writes triples. It must do the six things below.

Requirement	In plain words
Five predicates	Find all five predicates from the table above.
Use the parse	Use spaCy to split sentences and find names. At least **two** of your rules must use the **dependency parse** (how words connect), not only text patterns.
Pronouns	Turn "he" or "she" back into a name. Careful: in your private document one pronoun refers to a person who is *not* the main subject.
Negation	"He never studied at X" says something did *not* happen. It must not become a triple.
Same entity, one name	"Dr. Surname" and the full name are one person. Merge them, and report how many merges you made.
Provenance	Every triple must say which document and which sentence it came from.

Tip

Do not trust spaCy's entity labels completely. They can label an award as a person, or a city as an organisation. Look at your output, find these mistakes, and work around them. Starting with the private document is a good idea because it is short, so you can check every sentence by hand.

Done when: `out/triples.csv` exists with the columns `subject, predicate, object, source_doc, sentence`, and your README explains each rule in a few sentences.

Allowed and not allowed

Your extractor must be your own code. Do not use a ready-made relation-extraction model or a large language model to produce triples.

Step 3 – Rules for your graph (10 marks)

Create three files.

- `ontology.ttl`: classes `Person`, `Place`, `Organization`, `Award`; your five properties, each with a domain (what goes on the left) and range (what goes on the right); at least two `disjointWith` statements (for example, nothing is both a Person and a Place); and `bornIn` marked as functional (one value at most).
- `shapes.ttl`: SHACL shapes that check your *real* data (for example, "at most one birthplace").
- `shacl_report.txt`: the output of pySHACL when it validates your graph.

Then write a half-page note that does two things. First, for each violation in the report, say whether it is a real error in the data or a shape that is too strict. Second, name one check an OWL reasoner could do that SHACL cannot, and one check SHACL can do that a reasoner cannot (Week 5, Session B).

Done when: you have the three files and the note. Having violations is fine and expected; explaining them is the point.

Step 4 – Store and query (10 marks)

Use rdflib to turn your triples into `out/kg.ttl`, then load that file into your Fuseki dataset. Write six SPARQL queries and run them in Fuseki.

Query	What it must return
1	Every person with their birthplace, *including people with no birthplace* (hint: `OPTIONAL`).
2	How many triples there are for each predicate (hint: `GROUP BY`, `COUNT`).
3	Pairs of different people who share a birthplace.
4	People with *two or more different* birthplaces. Find them with a query, not by eye.
5	People with *no* `educatedAt` triple (hint: `FILTER NOT EXISTS`). Add one sentence on what this result can and cannot prove.
6	All of a person's doctoral ancestors, however many steps back (hint: a property path with `+`). If it returns nothing for your data, explain why.

Done when: you have `queries/q1.rq` to `q6.rq`, each result saved as CSV, and one screenshot of a query running in Fuseki.

Step 5 – Score your work (12 marks)

For each of your five Wikipedia people, get the Wikidata values for the same five predicates (Appendix C lists the property codes) and compare them with your triples. Make a table with, for each predicate:

Term	Meaning
True positive (TP)	You extracted it and Wikidata has it.
False positive (FP)	You extracted it but Wikidata does not have it.
False negative (FN)	Wikidata has it but you did not extract it.
Precision	TP / (TP + FP): of what you extracted, how much is right.
Recall	TP / (TP + FN): of what is there, how much you found.

Wikidata is not perfect. Sometimes your triple is correct and Wikidata is simply missing it.

Then look at your own mistakes. Your sheet lists positions. Sort your false positives alphabetically by (subject, predicate, object) and take the items at the positions on your sheet; do the same for the false negatives. If the list is shorter than a position, wrap around. For *each* chosen item write down: the sentence it came from, an error code from Appendix D, why it happened, and what would fix it. Finally, **fix at least two problems in your code** and show the scores before and after.

Done when: you have `eval/metrics.csv`, the Wikidata exports, and `eval/error_analysis.md`.

Step 6 – Predictions and reflection (3 marks)

Before you run your extractor on the whole collection for the first time, write `predictions.md` and commit it. Say: how many triples you expect for each predicate; which predicate you think will have the lowest precision, and why; and which difficulty in your private document you expect to fail on. After Step 5, write `reflection.md` (at most 400 words): compare your predictions with what happened, and describe one case where the open-world idea ("not in the graph does not mean false") changed how you read a result.

Do not skip the order

The commit history shows when you wrote your predictions. Writing them after the run earns nothing.

5 Your Folder at the End

```
kg-assignment-<rollno>/
  README.md            how to run it, and how each rule works
  data/raw/            five Wikipedia texts + private_document.txt
  src/extract.py       (and any other scripts)
  out/triples.csv      out/kg.ttl
  ontology.ttl   shapes.ttl   shacl_report.txt   (and your half-page note)
  queries/q1.rq ... q6.rq     results/q1.csv ... q6.csv
  eval/metrics.csv   eval/wikidata_*.csv   eval/error_analysis.md
  predictions.md   reflection.md   screenshots/
```

6 Marks

Part	Marks
Step 0 – Setup and evidence	5
Steps 1 and 2 – Text and extractor	20
Step 3 – Ontology, SHACL and note	10
Step 4 – Store and six queries	10
Step 5 – Scoring and error analysis	12
Step 6 – Predictions and reflection	3
Take-home	**60**
Viva	**40**
Total	**100**

7 Rules

Please read

This is individual work.

- **Not allowed:** using AI tools (chatbots, code assistants, AI autocomplete) to write your code, queries, ontology, analysis or reflection, or to extract triples for you.
- **Allowed:** course slides and handouts, and the official documentation of spaCy, rdflib, pySHACL, Apache Jena, Wikidata, SPARQL and SHACL. If you copy more than five lines from somewhere, say where from.
- Do not share your folder or your code with classmates. Similar code between students will be checked.

8 Common Questions

My Wikipedia triples look poor. Is that a problem?
No. Real text is messy. Good marks come from understanding why it fails and fixing a few causes, not from perfect scores.

A violation appears in my SHACL report.
Good. Decide whether the data is wrong or the shape is too strict, and write it in your note.

Query 6 returns nothing.
That can be correct if your data has no chain of advisors. Say why in your answer.

Can I use a regular expression instead of the parse?
Only for rules beyond the two parse-based ones the task requires.

Wikidata has no data for one of my people.
Use what exists, and write down what is missing and how it affects your scores.

A Install

```
pip install spacy rdflib pyshacl requests
python -m spacy download en_core_web_sm
```

B Fetching a Wikipedia article

```
import requests, pathlib

HEADERS = {"User-Agent": "KRR-coursework/1.0 (your-university-email)"}

def fetch(title):
    r = requests.get(
        "https://en.wikipedia.org/w/api.php",
        params={"action": "query", "prop": "extracts", "explaintext": 1,
                "redirects": 1, "titles": title, "format": "json"},
        headers=HEADERS, timeout=30)
    page = next(iter(r.json()["query"]["pages"].values()))
    return page["extract"]

pathlib.Path("data/raw").mkdir(parents=True, exist_ok=True)
text = fetch("Abdus Salam")
pathlib.Path("data/raw/Abdus_Salam.txt").write_text(text, encoding="utf-8")
```

Wikipedia and Wikidata ask automated programs to identify themselves, so keep the `User-Agent` line and put your own contact in it.

C Wikidata property codes

Your predicate	Wikidata property	Meaning
`bornIn`	P19	place of birth
`educatedAt`	P69	educated at
`receivedAward`	P166	award received
`worksAt`	P108	employer
`doctoralAdvisor`	P184	doctoral advisor

D Error codes for Step 5

Code	Type	Typical cause
E1	Wrong span	Name cut too short or too long
E2	Pronoun	A pronoun attached to the wrong person
E3	Alias split	One real entity became two nodes (or two became one)
E4	Direction or role	Subject and object swapped, or the wrong relation chosen
E5	Negation or time	A negated or past statement treated as current fact
E6	Source gap	Your triple is right but Wikidata lacks it
E7	Other	Explain it in your own words